Tic-Tac-Toe

Projektdokumentation

Java Swing • GUI-Anwendung • Modulare Architektur

Version 1.0 • Mai 2026 • Java 17+

Autoren: Fabio Stoffels, Gabriel de Souza leal, Peter Kretzer

Inhaltsverzeichnis

Inhaltsverzeichnis 2
1. Projektüberblick 3
1.1 Ziel des Projekts 3
1.2 Technische Rahmenbedingungen 3
1.3 Versionierung des Projekts 3
2. Projektstruktur 3
2.1 Ordnerstruktur 4
3. Architektur 4
3.1 Schichtenmodell 4
3.2 Kommunikationsfluss 4
3.3 Design-Entscheidungen 5
4. Klassen-Referenz 5
4.1 PlayerSymbol (Enum) 5
4.2 GameLogic 6
4.3 GameBoard 7
4.4 GameController 8
4.5 GameWindow 10
5. Spielablauf 11
5.1 Initialisierung 11
5.2 Spielzug 11
5.3 Gewinnerprüfung 11
5.4 Neustart 11
6. Kompilierung & Start 11
6.1 Voraussetzungen 11
6.2 Terminal-Befehle 12
6.3 Bekannte Einschränkungen 12
7. Erweiterungsmöglichkeiten 12
7.1 KI-Implementierung (Skizze) 13
8. Codequalität & Konventionen 13
8.1 Kommentarsprache 13
8.2 JavaDoc 13
8.3 OOP-Prinzipien 13
9. Vollständige Datei-Übersicht 14

1. Projektüberblick

Dieses Dokument beschreibt die Architektur, den Aufbau und die Funktionsweise des Tic-Tac-Toe-Projekts, das als Java-Anwendung mit grafischer Benutzeroberfläche (GUI) auf Basis von Swing umgesetzt wurde.

Auf einen Blick	
Sprache:	Java 17+
GUI-Bibliothek:	Java Swing (keine externen Abhängigkeiten)
Architektur:	Mehrschichtig – Logik, GUI und Controller getrennt
Dateien:	5 .java-Quelldateien
Designthema:	Catppuccin (dunkles Farbschema)

1.1 Ziel des Projekts

Das Projekt implementiert das klassische Zwei-Spieler-Brettspiel Tic-Tac-Toe als eigenständige Java-Desktop-Anwendung. Ziel war neben der Funktionalität auch eine saubere, objektorientierte Codestruktur mit klarer Trennung von Spiellogik und Präsentation.

1.2 Technische Rahmenbedingungen

- Java SE 17 oder höher (kompatibel mit Java 21 LTS)
- Ausschließlich Standard-Java-Bibliotheken (kein Maven, kein Gradle, keine Drittanbieter)
- Startbar mit: javac *.java && java Main
- Getestet unter Windows, macOS und Linux

1.3 Versionierung des Projekts

- Die Versionierung und somit auch die Speicherung läuft über GitHub, hier wird über Branche‘s das Projekt Stück für Stück aufgebaut.
- Link zum Repository: https://github.com/D-PEKR/Java_Repo

2. Projektstruktur

Das Projekt ist in fünf Quelldateien aufgeteilt, von denen jede eine klar abgegrenzte Aufgabe übernimmt. Diese Aufteilung folgt dem Prinzip der Separation of Concerns.

Datei	Zweck	Abhängigkeiten
Main.java	Einstiegspunkt, startet die GUI-Anwendung	GameWindow
PlayerSymbol.java	Enum für X, O und LEER	(keine)
GameLogic.java	Spielzustand, Züge, Gewinner-/Unentschiedenprüfung	PlayerSymbol
GameBoard.java	Swing-Panel mit 3×3-Buttons, Hover-Effekte	GameLogic, PlayerSymbol
GameController.java	ActionListener – verbindet GUI und Logik	GameLogic, GameBoard, GameWindow
GameWindow.java	JFrame – Hauptfenster mit Titel, Status, Neustart	GameBoard, GameController, PlayerSymbol

2.1 Ordnerstruktur

```
TicTacToe/
  Main.java           <- Programm-Einstieg
  PlayerSymbol.java   <- Enum (X | O | LEER)
  GameLogic.java      <- Spielregeln & Zustand
  GameBoard.java      <- Swing-Spielfeld (GUI)
  GameController.java <- Steuerlogik (MVC-Controller)
  GameWindow.java     <- Hauptfenster (MVC-View)
```

3. Architektur

Das Projekt folgt einer angepassten MVC-Architektur (Model–View–Controller). Die drei Schichten kommunizieren ausschließlich über klar definierte Methoden-Schnittstellen.

3.1 Schichtenmodell

Schicht	Klasse(n)	Beschreibung
Model	GameLogic, PlayerSymbol	Verwaltet den Spielzustand, kennt kein Swing
View	GameBoard, GameWindow	Zeigt den Zustand an, enthält keine Spielregeln
Controller	GameController	Verbindet Model und View, reagiert auf Klicks

3.2 Kommunikationsfluss

Der Ablauf eines einzelnen Spielzugs ist wie folgt:

1. Spieler klickt auf einen Button im GameBoard.
2. GameBoard löst ein ActionEvent mit ActionCommand "zeile,spalte" aus.
3. GameController empfängt das Event (actionPerformed).
4. GameController ruft GameLogic.zugAusfuehren(zeile, spalte) auf.
5. GameLogic prüft den Zug, aktualisiert das Brett und erkennt ggf. Spielende.
6. GameController ruft GameBoard.aktualisiereButton() auf.
7. Falls Spielende: GameController ruft GameWindow.zeigeSpielEnde() auf.

3.3 Design-Entscheidungen

- GameLogic hat keine Swing-Importe – sie kann isoliert getestet werden.
- GameBoard kennt keine Spielregeln – es ist ein reines Anzeige-Panel.
- Der ActionCommand codiert die Position als "zeile,spalte"-String, was die Buttons generisch hält.
- Das Hover-Verhalten wird per MouseAdapter simuliert, da Swing kein natives CSS-Hover kennt.

4. Klassen-Referenz

4.1 PlayerSymbol (Enum)

Aufzählungstyp für die drei möglichen Zustände einer Spielfeldzelle.

Konstante	Textdarstellung	Bedeutung
LEER	" "	Noch kein Zug auf dieser Zelle
X	"X"	Zelle belegt von Spieler X
O	"O"	Zelle belegt von Spieler O

Wichtige Methode: `andererSpieler()` – gibt das Symbol des Gegenspielers zurück (X→O, O→X).

```
public PlayerSymbol andererSpieler() {  1 usage  Peter Kretzer
    if (this == X) return O;
    if (this == O) return X;
    throw new IllegalStateException("LEER hat keinen Gegenspieler.");
```

Die Methode übernimmt folgende Aufgaben:

- Sie erkennt anhand des aktuellen Symbols, welcher Spieler als Nächstes am Zug ist.
- Sie verhindert ungültige Aufrufe, indem sie bei LEER eine Exception wirft.

- Sie stellt sicher, dass der Wechsel zwischen X und O konsistent und zentral gesteuert wird.

Vorteile dieser Lösung:

- Hohe Lesbarkeit: Der Code an anderen Stellen muss nicht wissen, wie der Spielerwechsel funktioniert.
- Fehlervermeidung: Ungültige Zustände werden frühzeitig abgefangen.
- Saubere Architektur: Die Spiellogik bleibt dort, wo sie hingehört – beim Symbol selbst.

Diese Methode trägt wesentlich zur Stabilität und Wartbarkeit des gesamten Spiels bei und ist ein gutes Beispiel für die sinnvolle Nutzung von Enums in Java.

4.2 GameLogic

Die Kernklasse für Spielzustand und Regelprüfung. Keine GUI-Abhängigkeit.

Methode	Rückgabetyp	Beschreibung
spielZuruecksetzen()	void	Setzt Brett und Spielerzustand auf Ausgangswert zurück
zugAusfuehren(zeile, spalte)	boolean	Setzt Symbol auf Zelle; false bei ungültigem Zug
pruefeGewinner(symbol)	boolean	Prüft alle Zeilen, Spalten und Diagonalen
istUnentschieden()	boolean	True wenn alle Felder belegt und kein Gewinner
getZelle(zeile, spalte)	PlayerSymbol	Gibt das Symbol der angegebenen Zelle zurück
getAktuellerSpieler()	PlayerSymbol	Gibt den gerade aktiven Spieler zurück
istSpielBeendet()	boolean	True nach Sieg oder Unentschieden

```java
public boolean zugAusfuehren(int zeile, int spalte) {  1 usage  Peter Kretzer
    // Ungültige Züge ablehnen
    if (spielBeendet) return false;
    if (brett[zeile][spalte] != PlayerSymbol.LEER) return false;

    // Zug setzen
    brett[zeile][spalte] = aktuellerSpieler;
    zugAnzahl++;

    // Spielende prüfen - nach diesem Zug wird kein Spielerwechsel mehr
    // benötigt, wenn das Spiel vorbei ist
    if (pruefeGewinner(aktuellerSpieler) || zugAnzahl == GITTER_GROESSE * GITTER_GROESSE) {
        spielBeendet = true;
    } else {
        // Spieler wechseln
        aktuellerSpieler = aktuellerSpieler.andererSpieler();
    }
```

Aufgaben der Methode:

- Validierung des Zugs: Es wird geprüft, ob das Spiel bereits beendet ist oder das gewählte Feld schon belegt wurde. Ungültige Züge werden konsequent abgelehnt.
- Setzen des Spielersymbols: Das Symbol des aktuellen Spielers wird in das entsprechende Feld eingetragen.
- Zugzähler aktualisieren: Die Methode erhöht die Anzahl der gesetzten Züge, was später für die Unentschieden-Erkennung wichtig ist.
- Gewinnerprüfung: Nach jedem Zug wird überprüft, ob der aktuelle Spieler gewonnen hat. Falls ja, wird das Spiel sofort beendet.
- Unentschieden-Erkennung: Wenn alle Felder belegt sind und kein Spieler gewonnen hat, wird das Spiel ebenfalls beendet.
- Automatischer Spielerwechsel: Nur wenn das Spiel *nicht* beendet ist, wird der Spieler gewechselt. Dadurch bleibt die Spiellogik klar strukturiert und fehlerfrei.

4.3 GameBoard

Swing-Panel (extends JPanel) mit einem GridLayout(3,3,8,8) aus 9 JButtons.

Methode	Beschreibung
setzeButtonListener(listener)	Registriert ActionListener an allen 9 Buttons
aktualisiereButton(zeile, spalte, symbol)	Setzt Text und Farbe eines Buttons
felderZuruecksetzen()	Setzt alle 9 Buttons auf LEER zurück
alleButtonsDeaktivieren()	Sperrt alle Buttons (nach Spielende)

Farbschema (Catppuccin Mocha):

- Hintergrund: #1E1E2E | Button: #313244 | Hover: #45475A

- Spieler X: #F38BA8 (Rosa) | Spieler O: #89DCEB (Hellblau)

```
public void setzeButtonListener(ActionListener listener) {  1 usage  Peter Kretzer
    for (int zeile = 0; zeile < GameLogic.GITTER_GROESSE; zeile++) {
        for (int spalte = 0; spalte < GameLogic.GITTER_GROESSE; spalte++) {
            // ActionCommand codiert die Position als "zeile,spalte"
            buttons[zeile][spalte].setActionCommand(zeile + "," + spalte);
            buttons[zeile][spalte].addActionListener(listener);
        }
    }
}
```

Aufgaben der Methode:

- Registrierung eines ActionListeners: Der Controller übergibt einen Listener, der auf alle Button-Klicks reagiert.
- Codierung der Position: Jeder Button erhält einen ActionCommand im Format "zeile,spalte". Dadurch weiß der Controller genau, welches Feld angeklickt wurde, ohne dass die GUI selbst Spiellogik kennen muss.
- Entkopplung von GUI und Logik: Die GUI führt keine Spielregeln aus und trifft keine Entscheidungen. Sie meldet nur Ereignisse weiter — ein sauberes MVC-ähnliches Design.

4.4 GameController

Implementiert ActionListener. Wertet den ActionCommand der Buttons aus und koordiniert Logik und GUI.

```
// ActionCommand-Format: "zeile,spalte"
String[] teile = event.getActionCommand().split(",");
int zeile  = Integer.parseInt(teile[0]);
int spalte = Integer.parseInt(teile[1]);
```

```java
@Override  Peter Kretzer
public void actionPerformed(ActionEvent event) {
    // Position aus dem ActionCommand parsen
    String[] teile = event.getActionCommand().split( regex: ",");
    int zeile  = Integer.parseInt(teile[0]);
    int spalte = Integer.parseInt(teile[1]);

    // Zug in der Logik ausführen - schlägt fehl, wenn Feld belegt ist
    boolean zugGueltig = logik.zugAusfuehren(zeile, spalte);

    if (!zugGueltig) {
        return; // Ungültiger Zug - nichts tun
    }

    // Button-Darstellung aktualisieren
    spielfeld.aktualisiereButton(zeile, spalte, logik.getZelle(zeile, spalte));

    // Spielende prüfen und entsprechend reagieren
    if (logik.istSpielBeendet()) {
        spielfeld.alleButtonsDeaktivieren();

        if (logik.istUnentschieden()) {
            // Unentschieden
            fenster.zeigeSpielEnde( gewinner: null);
        } else {
            // Gewinner ist der zuletzt aktive Spieler (noch nicht gewechselt)
            fenster.zeigeSpielEnde(logik.getAktuellerSpieler());
        }
    } else {
        // Statuszeile im Fenster aktualisieren
        fenster.aktualisiereStatus(logik.getAktuellerSpieler());
    }
}
```

Aufgaben der Methode actionPerformed()

- Auslesen der Benutzeraktion: Jeder Button im Spielfeld besitzt einen ActionCommand im Format "zeile,spalte". Der Controller liest diese Information aus und weiß dadurch genau, welches Feld angeklickt wurde.
- Weitergabe an die Spiellogik: Der Controller ruft zugAusfuehren() auf und überlässt der Logik die Entscheidung, ob der Zug gültig ist.
- Aktualisierung der GUI: Nach einem gültigen Zug wird der entsprechende Button im GameBoard aktualisiert (Symbol + Farbe + Deaktivierung).
- Spielende erkennen: Der Controller prüft über die Logik, ob das Spiel vorbei ist:
    - Bei einem Sieg wird der Gewinner an das Hauptfenster gemeldet.
    - Bei einem Unentschieden wird ebenfalls eine Meldung ausgegeben.

    - Alle Buttons werden deaktiviert, um weitere Eingaben zu verhindern.
- Statusanzeige aktualisieren: Wenn das Spiel weiterläuft, wird im Hauptfenster angezeigt, welcher Spieler als Nächstes am Zug ist.

4.5 GameWindow

Hauptfenster (extends JFrame) mit vier Bereichen:

- Kopfzeile: Spieltitel auf dunklem Hintergrund
- Mitte: Spielfeld-Wrapper mit Innenabstand
- Fusszeile: Statuslabel + Neustart-Button
- Dialoge: JOptionPane nach Spielende

Das Fenster ist auf 480×580 Pixel fixiert (setResizable(false)) und wird beim Start auf dem Bildschirm zentriert (setLocationRelativeTo(null)).

```
public void zeigeSpielEnde(PlayerSymbol gewinner) {  2 usages  Peter Kretzer
    // Nachricht formulieren
    String nachricht;
    if (gewinner == null) {
        nachricht = "Unentschieden! Niemand gewinnt.";
        statusLabel.setText("Unentschieden!");
        statusLabel.setForeground(FARBE_STATUS);
    } else {
        nachricht = "Spieler " + gewinner.getAnzeige() + " hat gewonnen!";
        statusLabel.setText("Spieler " + gewinner.getAnzeige() + " gewinnt!");
        statusLabel.setForeground(gewinner == PlayerSymbol.X ? FARBE_X : FARBE_O);
    }
```

Aufgaben der Methode zeigeSpielEnde()

- Erkennen des Spielausgangs: Die Methode erhält vom Controller entweder den Gewinner oder null bei einem Unentschieden.
- Aktualisierung der Statusleiste: Der Statustext wird sofort angepasst:
    - Bei einem Sieg wird der Gewinner farblich hervorgehoben.
    - Bei einem Unentschieden wird ein neutraler Text angezeigt.
- Anzeige eines Dialogfensters: Ein JOptionPane informiert den Spieler über das Ergebnis und bietet direkt die Möglichkeit, ein neues Spiel zu starten.
- Neustart des Spiels: Wenn der Benutzer „Ja“ wählt, wird über den Controller ein vollständiger Neustart ausgelöst:
    - Spiellogik wird zurückgesetzt
    - Spielfeld wird geleert
    - Statusanzeige wird aktualisiert

5. Spielablauf

5.1 Initialisierung

8. Main.java ruft SwingUtilities.invokeLater(() -> new GameWindow()) auf.
9. GameWindow erstellt GameBoard, GameLogic und GameController.
10. GameController registriert sich als ActionListener an den 9 Buttons.
11. Das Fenster erscheint zentriert auf dem Bildschirm, Spieler X beginnt.

5.2 Spielzug

12. Spieler klickt auf ein leeres Feld.
13. GameLogic prüft den Zug (Feld leer? Spiel nicht beendet?).
14. Bei gültigem Zug: Symbol wird gesetzt, Zelle gesperrt, Gewinnerprüfung erfolgt.
15. Statuslabel wird aktualisiert; bei Spielende erscheint ein Dialog.

5.3 Gewinnerprüfung

Die Methode pruefeGewinner() durchläuft nach jedem Zug:

- 3 Zeilen (jeweils alle 3 Zellen gleich)
- 3 Spalten (jeweils alle 3 Zellen gleich)
- Hauptdiagonale ([0,0] [1,1] [2,2])
- Nebendiagonale ([0,2] [1,1] [2,0])

Sind alle 9 Felder belegt und kein Symbol hat gewonnen, meldet istUnentschieden() true.

5.4 Neustart

Über den "Neues Spiel"-Button oder den Ja-Button im Endscreen wird controller.neuesSpiel() aufgerufen:

- GameLogic.spielZuruecksetzen() leert das interne Brett.
- GameBoard.felderZuruecksetzen() aktiviert alle Buttons und entfernt Symbole.
- Das Statuslabel zeigt wieder "Spieler X ist dran".

6. Kompilierung & Start

6.1 Voraussetzungen

- Java Development Kit (JDK) 17 oder neuer
- Kein Build-Tool erforderlich (kein Maven/Gradle)
- Kein Internet / keine externen Bibliotheken

6.2 Terminal-Befehle

Alle fünf .java-Dateien müssen im selben Verzeichnis liegen:

```
# Schritt 1: In den Projektordner wechseln
cd TicTacToe/

# Schritt 2: Alle Quelldateien kompilieren
javac *.java

# Schritt 3: Programm starten
java Main
```

Hinweis zu Java-Versionen

Das Projekt wurde mit Java 21 LTS entwickelt und getestet.

Es ist abwärtskompatibel bis Java 17 (keine neueren Sprachfeatures).

Ältere Versionen (Java 8-16) werden nicht offiziell unterstützt.

6.3 Bekannte Einschränkungen

- Kein KI-Gegner: Das Spiel erfordert zwei menschliche Spieler.
- Kein Highscore-Speicher: Spielstände werden nicht persistiert.
- Fenster nicht skalierbar: Feste Größe von 480×580 Pixel.

7. Erweiterungsmöglichkeiten

Das modulare Design erleichtert die spätere Erweiterung des Projekts erheblich. Da Logik und GUI vollständig getrennt sind, können Verbesserungen isoliert eingeführt werden.

Erweiterung	Betroffene Klasse(n)	Aufwand
KI-Gegner (Minimax-Algorithmus)	GameLogic + GameController	Hoch
Spielstand-Anzeige (Siege/Niederlagen)	GameWindow + GameController	Niedrig
Netzwerkspiel (Client-Server)	Neue Klasse NetworkHandler	Sehr hoch
Einstellungen (Spielfeldgröße 4×4)	GameLogic + GameBoard	Mittel
Sound-Effekte bei Zügen	Neue Klasse SoundManager	Niedrig
Animation (Gewinnlinie hervorheben)	GameBoard	Mittel

7.1 KI-Implementierung (Skizze)

Ein Minimax-Algorithmus könnte direkt in GameLogic eingebettet werden, da dort das komplette Spielfeld verfügbar ist:

```
// In GameLogic - neuer KI-Zug
public int[] berechneKIZug() {
    int bestesBewertung = Integer.MIN_VALUE;
    int[] besterZug = {-1, -1};
    for (int z = 0; z < 3; z++) {
        for (int s = 0; s < 3; s++) {
            if (brett[z][s] == PlayerSymbol.LEER) {
                brett[z][s] = PlayerSymbol.O; // KI spielt O
                int score = minimax(false, 0);
                brett[z][s] = PlayerSymbol.LEER;
                if (score > bestesBewertung) {
                    bestesBewertung = score;
                    besterZug = new int[]{z, s};
                }
            }
        }
    }
    return besterZug;
}
```

8. Codequalität & Konventionen

8.1 Kommentarsprache

Alle Kommentare (JavaDoc und Inline) sind auf Deutsch verfasst. Bezeichner (Klassen, Methoden, Variablen) folgen den üblichen Java-Konventionen in Englisch, um maximale IDE-Kompatibilität zu gewährleisten.

8.2 JavaDoc

Alle öffentlichen Klassen und Methoden sind mit JavaDoc-Kommentaren versehen. Jede Klasse enthält:

- Eine einleitende Beschreibung der Verantwortlichkeiten
- @param-Tags für alle Parameter
- @return-Tags für Rückgabewerte
- @throws-Tags wo relevant

8.3 OOP-Prinzipien

Prinzip	Umsetzung im Projekt
Single Responsibility	Jede Klasse hat genau eine Aufgabe
Separation of Concerns	GameLogic kennt kein Swing; GameBoard kennt keine Regeln

Prinzip	Umsetzung im Projekt
Kapselung	Alle Felder private, Zugriff nur über Methoden
Polymorphismus	ActionListener-Interface ermöglicht flexible Ereignisverarbeitung

9. Vollständige Datei-Übersicht

Alle fünf Quelldateien in der Übersicht mit Zeilenzahl und Schlüsselelementen:

Datei	ca. Zeilen	Schlüsselelemente
Main.java	~35	main(), SwingUtilities.invokeLater
PlayerSymbol.java	~60	Enum-Konstanten, andererSpieler(), getAnzeige()
GameLogic.java	~130	brett[][], zugAusfuehren(), pruefeGewinner()
GameBoard.java	~140	JButton[][], MouseAdapter, Farbkonstanten
GameController.java	~70	actionPerformed(), neuesSpiel()
GameWindow.java	~200	JFrame, JLabel, BorderLayout, JOptionPane

Ende der Dokumentation • Tic-Tac-Toe v1.0 • Mai 2026